Manual de despliegue y portabilidad

Guía de ejecución con Docker Compose y propuesta hipotética de despliegue en la nube

1. Objetivo y alcance

Documentar la reproducción de una aplicación académica de clasificación de riesgo crediticio que integra una interfaz Streamlit, una API FastAPI y un modelo Random Forest previamente entrenado. La validación operativa se efectuó en GitHub Codespaces (Linux remoto); no se instaló Docker en la laptop corporativa.

Alcance de uso: demostración académica. El modelo conserva un dictamen NO-GO para decisiones bancarias reales. No se utiliza información confidencial de clientes.

2. Componentes y arquitectura

Componente	Función / comunicación
Streamlit (frontend)	Captura 20 variables y realiza POST hacia /predict.
FastAPI (backend)	Valida campos y categorías, ejecuta inferencia y devuelve JSON.
Random Forest (joblib)	Modelo reproducido desde South German Credit; umbral 0.28.
Docker Compose	Dos servicios en red interna; salud y dependencia gestionadas por Compose.

Ruta de inferencia: navegador → Streamlit → HTTP (red privada Docker) → FastAPI → modelo → respuesta JSON → visualización.

3. Requisitos técnicos

- Docker Engine y Docker Compose con soporte de contenedores Linux; acceso a las imágenes de Python y PyPI durante la primera construcción.
- Memoria: referencia de configuración de 2 GB para la API y 1 GB para Streamlit; disponibilidad de CPU y almacenamiento suficiente.
- Puertos 8765 (API) y 8766 (interfaz) disponibles en el anfitrión; definidos exclusivamente en 127.0.0.1.
- No se requieren tarjetas ni despliegues en la nube para reproducir la validación documentada; el uso de Codespaces sí consume su cuota de cómputo.

4. Despliegue de extremo a extremo

4.1. Obtener y levantar el proyecto

```
git clone https://github.com/jalfredolarrea-debug/Fase2-Despliegue-Riesgo-Crediticio.git
cd Fase2-Despliegue-Riesgo-Crediticio
docker compose config -q
docker compose up -d --build
docker compose ps
```

Criterio de aceptación: los servicios api y frontend deben mostrarse en ejecución, idealmente como healthy. La construcción puede descargar dependencias en el primer uso.

4.2. Verificar backend, frontend y predicción

```
curl -fsS http://127.0.0.1:8765/health
curl -fsS http://127.0.0.1:8766/_stcore/health
python tests/test_integracion.py
```

Respuesta esperada de /health: estado "operativo", modelo "Random Forest" y 20 variables. Para Streamlit se espera "ok". Las pruebas reproducen 200 predicciones y rechazan entradas inválidas con HTTP 422.

Abrir la interfaz en http://127.0.0.1:8766; en Codespaces, abrir el puerto reenviado 8766 desde la pestaña Puertos y mantenerlo en modo privado. La documentación OpenAPI está disponible en /docs del puerto 8765.

4.3. Reconstrucción del modelo (opcional)

```
docker build -f Dockerfile.train -t fase2-entrenamiento .
docker run --rm -v "$PWD/artifacts:/app/artifacts" fase2-entrenamiento
```

Este procedimiento reproduce el entrenamiento, contrasta las probabilidades y la matriz de confusión con los archivos de referencia, y solo guarda el artefacto si pasa las verificaciones. La sintaxis del volumen mostrado corresponde a una shell Linux.

4.4. Detener los servicios

```
docker compose down
```

Cerrar el navegador no detiene necesariamente un Codespace. Al finalizar, detenerlo desde la interfaz de GitHub para evitar consumo innecesario de cuota.

5. Evidencia local y criterios de portabilidad

El 2 de noviembre de 2026 se registraron las siguientes comprobaciones dentro del Codespace: construcción de imágenes propias, contenedores saludables, consumo exitoso de la API por Streamlit y reproducción de 200 predicciones con validación de entradas. La publicación en GitHub se verificó mediante git push y seguimiento main...origin/main.

El 3 de noviembre de 2026 se clonó el repositorio público en una carpeta independiente. Docker Compose levantó api y frontend en estado healthy y las pruebas reprodujeron 200 predicciones y rechazaron entradas inválidas. Constan en evidence/auditoria_reproducibilidad.txt y la captura 03. Fue una prueba remota en Codespaces, no una ejecución física en otra computadora.

6. Estrategia propuesta de despliegue en nube — no ejecutada

Se plantea Google Cloud Run como alternativa de arquitectura, únicamente para documentar su viabilidad. No se creó ningún proyecto, servicio de nube o cuenta de facturación; su ejecución no forma parte de las evidencias.

6.1. Adaptación de arquitectura

El esquema local usa el nombre Docker Compose "api" como host interno. Ese nombre no es automáticamente válido en Cloud Run. Para mantener ambos contenedores en una única instancia multicontenedor, se propone Streamlit como contenedor de ingreso y FastAPI como contenedor

auxiliar, con API_URL=http://127.0.0.1:8000. Se configuraría el puerto de ingreso 8501 y condiciones de arranque/salud. Esta topología requiere validación técnica en nube antes de considerarse lista para producción.

6.2. Procedimiento hipotético

1. Obtener autorización institucional, definir proyecto/identidad/IAM, presupuesto y controles de gasto; no avanzar si exige un medio de pago.

2. Fijar versiones e imágenes reproducibles; construir imágenes de FastAPI y Streamlit por separado y publicarlas en Artifact Registry autorizado.

3. Configurar Cloud Run multicontenedor: Streamlit como ingreso en 8501 y FastAPI como auxiliar en 8000; definir API_URL local, memoria, CPU y revisiones de imagen.

4. Restringir el acceso con autenticación y controles de ingreso. Evitar publicar una aplicación de evaluación crediticia para uso abierto.

5. Comprobar /health, /predict, interfaz, registros y pruebas de casos extremos; documentar la revisión y un mecanismo de reversión.

6. Supervisar consumo, errores, latencia y límites de utilización; establecer proceso para deshabilitar/eliminar el servicio si procediera.

6.3. Condiciones y riesgos

Riesgo	Medida propuesta
Gasto o facturación	No iniciar despliegue sin evaluación económica y aprobación formal.
Exposición de resultados	Identidad y acceso restringidos; sin datos reales de clientes.
Incompatibilidad de puertos	Configurar puerto de ingreso y comunicación local multicontenedor.
Desempeño estadístico insuficiente	Conservar NO-GO; restringir uso a evidencia académica.

7. Referencias y trazabilidad

7.1. Herramientas de apoyo documental

Se utilizó ChatGPT como apoyo para estructurar código, pruebas y documentación. Sus sugerencias se contrastaron con la ejecución en Codespaces, las métricas de la Actividad 6 y los archivos publicados en GitHub. Los resultados y el dictamen NO-GO proceden de estas verificaciones, no de afirmaciones de la herramienta.

Repositorio: https://github.com/jalfredolarrea-debug/Fase2-Despliegue-Riesgo-Crediticio | Datos: South German Credit, UCI (DOI 10.24432/C5QG88).

Guías técnicas: Google Cloud Run, https://docs.cloud.google.com/run/docs/deploying y https://docs.cloud.google.com/run/docs/configuring.